Spring Security



UP ASI
Bureau E204

PLAN DU COURS

-Introduction

-Configuration de Spring Security

-Mise en place de Spring Security

-TP

INTRODUCTION

- **Spring Security** est un Framework de sécurité léger qui fournit une authentification et un support d'autorisation afin de sécuriser les applications Spring.

- Spring Security répond à deux questions majeures :

- Qui peut accéder à l'application ? ➔ **Authentification**
- Est-ce que l'utilisateur connecté a le droit d'accéder à cette ressource ? ➔ **Autorisation** permettant ou pas à un utilisateur d'accéder à une ressource.

Configuration de Spring Security

- Pour accéder à une application, nous pouvons généralement procéder de deux façons :

- La connexion statique: En utilisant des données statiques (en mémoire ou dans un fichier).
- La connexion dynamique: En utilisant des données dynamiques provenant d'une base de données.

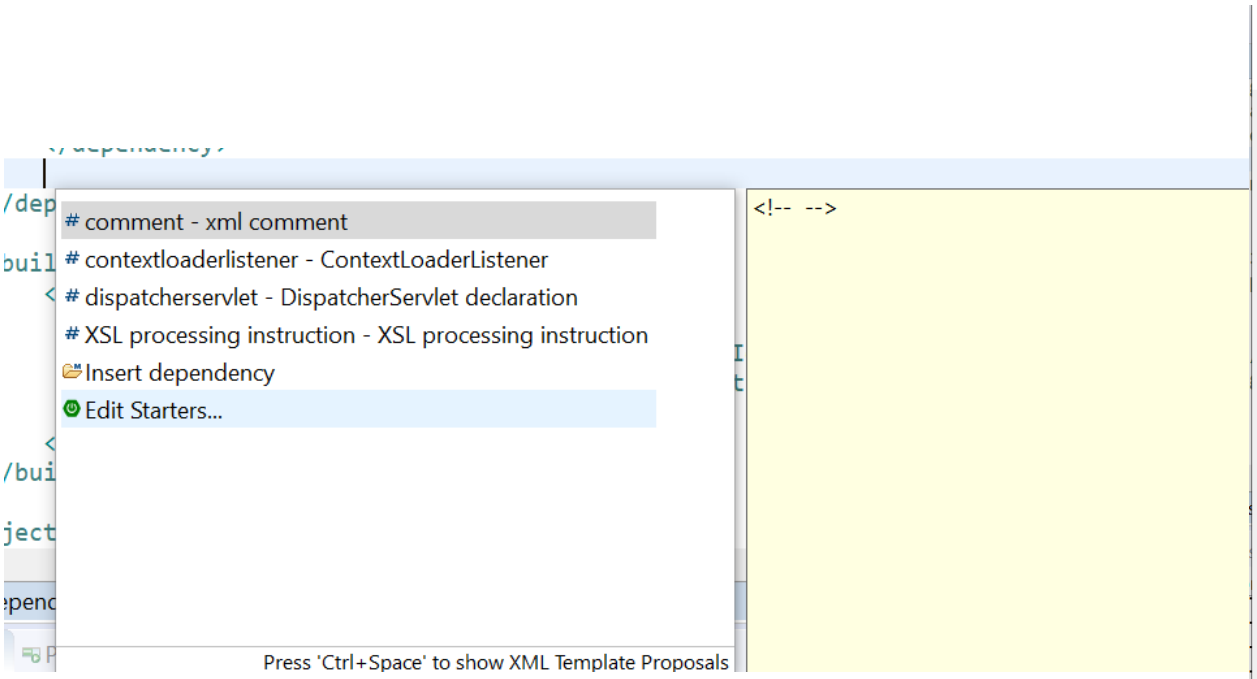
➔ Pour notre cours, nous allons opter pour la connexion dynamique

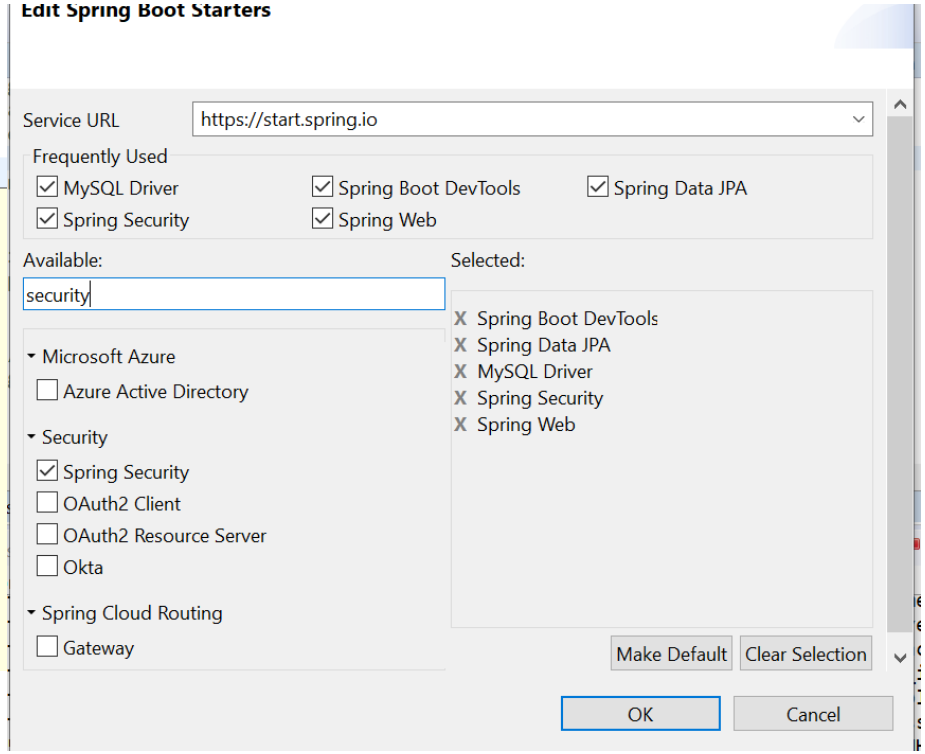
Configuration de Spring Security - DÉPENDANCES

- Pour configurer la partie sécurité au niveau de notre projet Spring Boot, il suffit d'aller dans le pom.xml et d'ajouter les dépendances Spring Security (ou le starter spring Security) :

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.1</version>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>
</dependency>
    <dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Configuration de Spring Security - DÉPENDANCES





- Il suffit par la suite de faire un clean install et maven update de votre projet.

Spring Security- Création des entités User et Role

- Création des entités:
  - Pour gérer l'authentification et l'accès aux différentes fonctionnalités, il suffit de créer deux entités User et Role
  - Nous pouvons profiter d'une entité déjà existante pour l'authentification (entité opérateur dans le cas de notre projet)
  - Chaque user a ses propres attributs pour se connecter (username,password,etc…).
  - Chaque user peut avoir le rôle d'ADMIN ou SUPERADMIN ou autres (CLIENT,etc…) .
  - Pour chaque rôle, l'utilisateur peut accéder à certaines fonctionnalités et sera interdit d'accéder à d'autres.
  - L'association entre user et rôle est ManyToMany

Spring Security- Création des entités User et Role

- Création des entités:
  - Créer les deux entités Operateur et Role
  - Association: ManyToMany

```
@Entity @Getter @Setter
@AllArgsConstructor
@NoArgsConstructor
public class Role  implements Serializable {
class User   implements Serializable {
          @Id
    @GeneratedValue(strategy =
GenerationType.AUTO)
private int id;
@Enumerated(EnumType.STRING)
private RoleName role;
```

```
import org.springframework.security.core.GrantedAuthority;
public enum RoleName implements
GrantedAuthority {
ADMIN,
SUPERADMIN;
OPERATEUR;
  @Override
  public String getAuthority() {
    return "ROLE_" + name();
  }}
```

Spring Security- Création des entités User et Role

```
public class Operateur implements Serializable {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long idOperateur;
private String nom;
private String prenom;
private String email;
private String username;
private String password;
@ManyToMany(fetch = FetchType.LAZY)
@JoinTable(name = "operateur_roles", joinColumns = @JoinColumn(name =
"operateur_id"),
inverseJoinColumns = @JoinColumn(name = "role_id"))
private Set<Role> roles = new HashSet<>();
@OneToMany @JsonIgnore
private Set<Facture> factures;
```

Spring Security- Configuration de la couche Repository

- Dans la couche repository, créer les deux interfaces Repository respectifs aux entités Operateur et Role:

  OperateurRepository et RoleRepository.
- Nous allons utiliser les deux fonctionnalités basées sur des **keywords** findByUsername et findByRole.

```
@Repository
public interface OperateurRepository extends CrudRepository<Operateur, Long> {
Optional<Operateur> findByUsername(String username);
Boolean existsByUsername(String username);
Boolean existsByEmail(String email);
}
@Repository
public interface RoleRepository extends JpaRepository<Role, Long> {
Optional<Role> findByName(ERole name);
}
```

Spring Security- Configuration de la couche Service

Dans la couche service, créer la classe service **`AuthTokenFilter`**

- Implémente les méthodes permettant de retourner des informations sur l'utilisateur connecté dans la base de données.
- Un objet de classe UserDetails est créé à partir de l'utilisateur et envoyé au fichier de configuration de sécurité

- La méthode **loadUserByUsername**() sera invoquée par Spring Security lors de l'authentification des utilisateurs.
- La méthode renvoie aussi un ensemble de rôles (autorités) à utiliser par Spring Security dans le processus d'autorisation.

Configuration de la couche Service

```
public class AuthTokenFilter extends OncePerRequestFilter {
@Autowired private JwtUtils jwtUtils;
@Autowired private UserDetailsServiceImpl userDetailsService;
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain filterChain) throws ServletException, IOException {
try { String jwt = parseJwt(request);
if (jwt != null && jwtUtils.validateJwtToken(jwt)) {
String username = jwtUtils.getUserNameFromJwtToken(jwt);
UserDetails userDetails = userDetailsService.loadUserByUsername(username);
UsernamePasswordAuthenticationToken authentication = new UsernamePasswordAuthenticationToken(
userDetails, null, userDetails.getAuthorities());
authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
SecurityContextHolder.getContext().setAuthentication(authentication);
}} catch (Exception e) {
logger.error("Cannot set user authentication: {}", e); }
filterChain.doFilter(request, response);
}
```

Configuration de la couche Service

- Dans la couche Service, créer la classe UserDetailsServiceImpl qui contient les méthodes:
  - loadUserByUsername

- **UserDetailsServiceImpl** implémente donc une méthode vérifiant l'existence d'un utilisateur selon la valeur de username (en utilisant le repository).

Configuration de la couche Service: La classe UserService

```
@Service
public class UserDetailsServiceImpl implements UserDetailsService {
  @Autowired
  OperateurRepository operateurRepository;
  @Override
  @Transactional
  public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
    Operateur operateur = operateurRepository.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException("Operateur Not found
  with username: " + username));

    return UserDetailsImpl.build(operateur);
  }
}
```

Spring Security- Configuration de la classe SecurityConfig

- Dans la couche sécurité (dans un package nommé security), créer la classe WebSecurityConfig (elle sert à configurer la sécurité de l'application):

  - Annotée **@EnableWebSecurity** pour activer la prise en charge de la sécurité Web de Spring Security.
  - La méthode **configure(HttpSecurity**) définit quels chemins d'URL doivent être sécurisés et lesquels ne le doivent pas (le chemin /registration est configuré pour ne nécessiter aucune authentification. Tous les autres chemins doivent être authentifiés).

Spring Security- Configuration de la classe WebSecurityConfig

```
@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true)
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {
@Autowired
UserDetailsServiceImpl userDetailsService;
@Autowired
private AuthEntryPointJwt unauthorizedHandler;
……
@Override
protected void configure(HttpSecurity http) throws Exception {
http.cors().and().csrf().disable().exceptionHandling().authenticationEntryPoint(unauthorizedHandler).and()
sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS).and().authorizeRequests()
antMatchers("/api/auth/**").permitAll().
antMatchers("/operateur/**").permitAll().
antMatchers("/pourcentageRecouvrement/**").permitAll()
anyRequest().authenticated();
http.addFilterBefore(authenticationJwtTokenFilter(), UsernamePasswordAuthenticationFilter.class); }}
```

L'ordre est important !

Spring Security- Configuration de la classe WebSecurityConfig

Configuration	Signification
La méthode configure(HttpSecurity http)	Configuration des utilisateurs authentifiés et autorisés
La méthode configure(AuthenticationManagerBuilder auth)	Dans laquelle Spring Security chargera les détails de l'utilisateur pour effectuer l'authentification et l'autorisation à travers l'implémentation de l'interface UserDetailsService
@EnableWebSecurity	pour activer la prise en charge de la sécurité Web de Spring Security et fournir l'intégration Spring MVC
.antMatchers("/operateur").permitAll()	La page /registration ne demande pas une authentification
.antMatchers("/pourcentageRecouvrement/**").access("hasRole ('SUPERADMIN')")	La page / getRevenuBrutProduit est accessible par le role superadmin
.antMatchers("/retrieve-all-operateurs").access("hasRole ('ADMIN')")	La page /retrieve-all-operateurs est accessible par le role admin
.passwordEncoder(bCryptPasswordEncoder);	Pour encoder le mot de passe
.csrf()	Utilisé pour empêcher la falsification de requêtes intersites, la protection CSRF est activée (par défaut).

Spring Security- Configuration de la couche Controller

- Il suffit par la suite d'exposer le service de logIn (registration)/logUp (authentification) :

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {
@Autowired AuthenticationManager authenticationManager;
@Autowired OperateurRepository operateurRepository;
@Autowired RoleRepository roleRepository;
@Autowired PasswordEncoder encoder;
@Autowired JwtUtils jwtUtils;
public ResponseEntity<?> authenticateUser(@Valid @RequestBody LoginRequest loginRequest)
{
Authentication authentication = authenticationManager.authenticate(
new UsernamePasswordAuthenticationToken(loginRequest.getUsername(),loginRequest.getPassword()));

SecurityContextHolder.getContext().setAuthentication(authentication);
String jwt = jwtUtils.generateJwtToken(authentication);

UserDetailsImpl userDetails = (UserDetailsImpl) authentication.getPrincipal();
List<String> roles = userDetails.getAuthorities().stream()
.map(item -> item.getAuthority()).collect(Collectors.toList());

return ResponseEntity.ok(new JwtResponse(jwt, userDetails.getId(),
userDetails.getUsername(),userDetails.getEmail(), roles));
}
```

Spring Security- Configuration de la couche Controller

```
@PostMapping("/signup")
public ResponseEntity<?> registerUser(@RequestBody SignupRequest signUpRequest) {
if (operateurRepository.existsByUsername(signUpRequest.getUsername())) {
return ResponseEntity.badRequest().body(new MessageResponse("Error: Username is already taken!"));}

if (operateurRepository.existsByEmail(signUpRequest.getEmail())) {
return ResponseEntity.badRequest().body(new MessageResponse("Error: Email is already in use!"));}
Operateur operateur = new Operateur(signUpRequest.getUsername(), signUpRequest.getEmail(),
encoder.encode(signUpRequest.getPassword()));
Set<String> strRoles = signUpRequest.getRole(); Set<Role> roles = new HashSet<>();
if (strRoles == null) {
Role userRole = roleRepository.findByName(ERole.ROLE_CLIENT)
.orElseThrow(() -> new RuntimeException("Error: Role is not found."));
roles.add(userRole);}
else { strRoles.forEach(role -> { switch (role) {
case "ROLE_ADMIN":
Role adminRole = roleRepository.findByName(ERole.ROLE_ADMIN)
.orElseThrow(() -> new RuntimeException("Error: Role is not found."));
roles.add(adminRole);
break;
default:
Role userRole = roleRepository.findByName(ERole.ROLE_OPERATEUR)
.orElseThrow(() -> new RuntimeException("Error: Role is not found."));
roles.add(userRole);
}}); }
operateur.setRoles(roles); operateurRepository.save(operateur);
return ResponseEntity.ok(new MessageResponse("User registered successfully!")); }
```

TP –Ajout Admin et User

POST http://localhost:8089/SpringMVC/api/auth/signup **Send**

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
    "username":"admin",
    "email":"admin@gmail.tn",
    "password":"admin",
    "role": ["ROLE_ADMIN"]
}
```

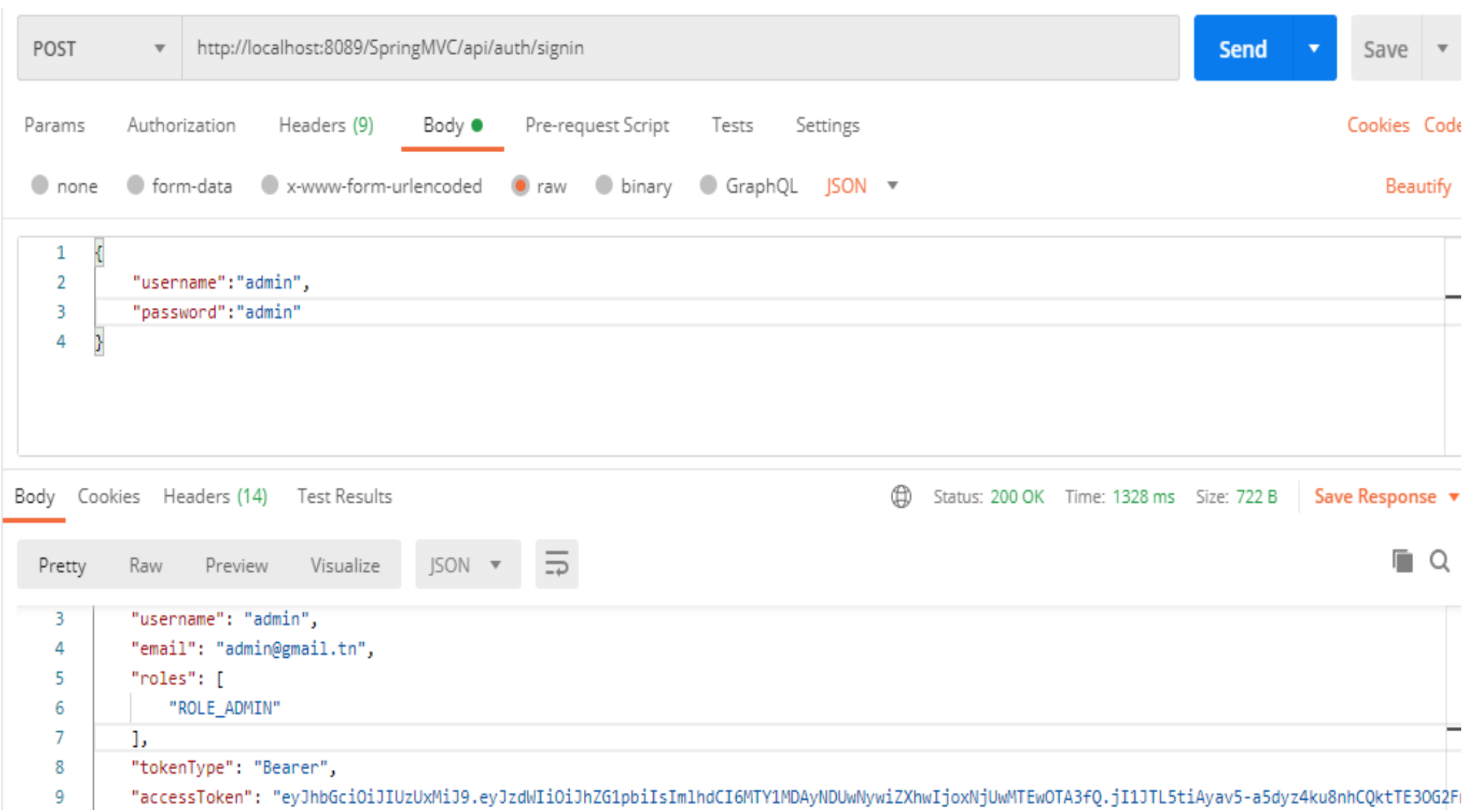
Body Cookies Headers (14) Test Results Status: 200 OK Time: 308 ms Size: 478 B Save

Pretty Raw Preview Visualize JSON

```
{
    "message": "User registered successfully!"
}
```

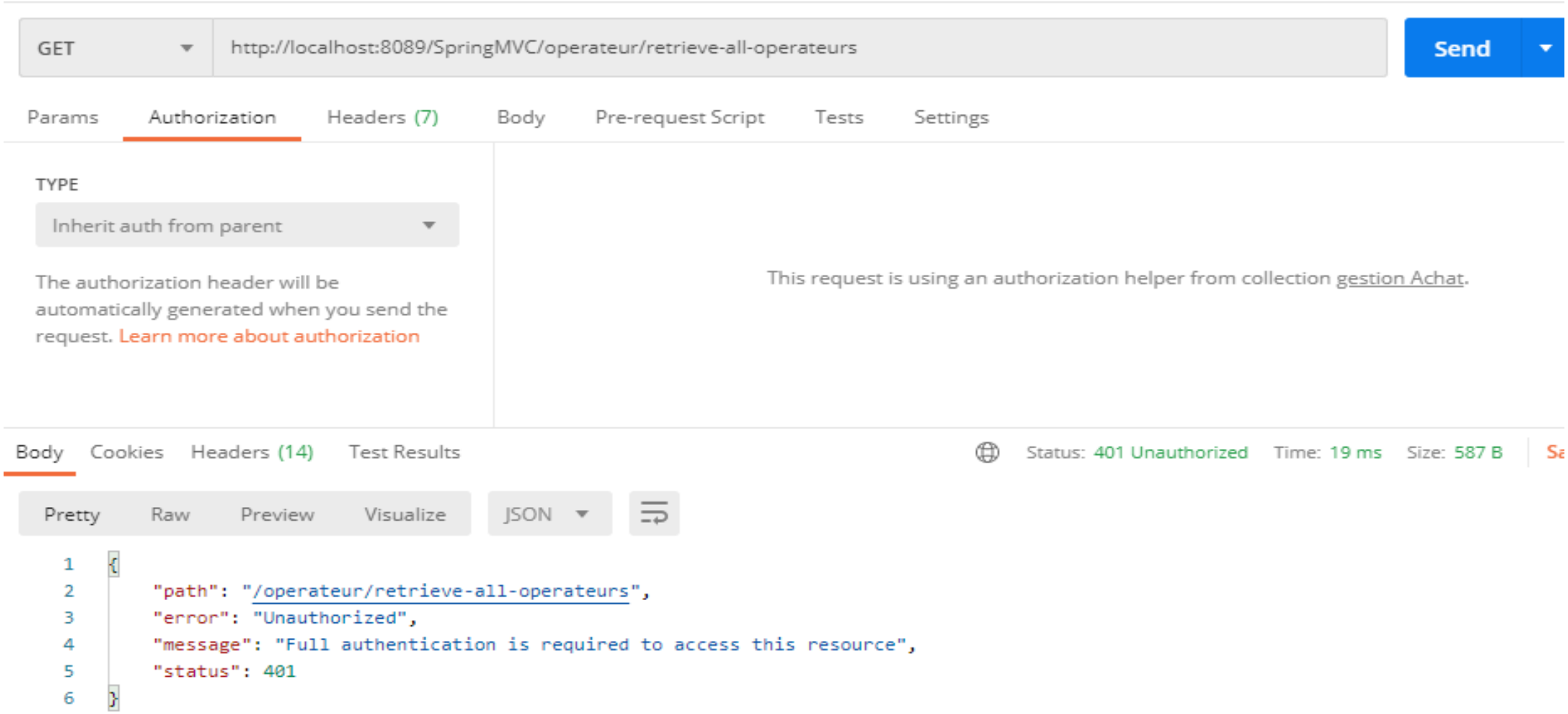
	id_operateur	nom	password	prenom	email	login	username
Éditer Copier Supprimer	3	admin	$2a$10$Rzh4mAHkQJRQxXGZPHGMdOEd3alMIPZuiNQ86pPNeoN...	admin	admin@gmail.tn	admin	admin

TP –Authentification



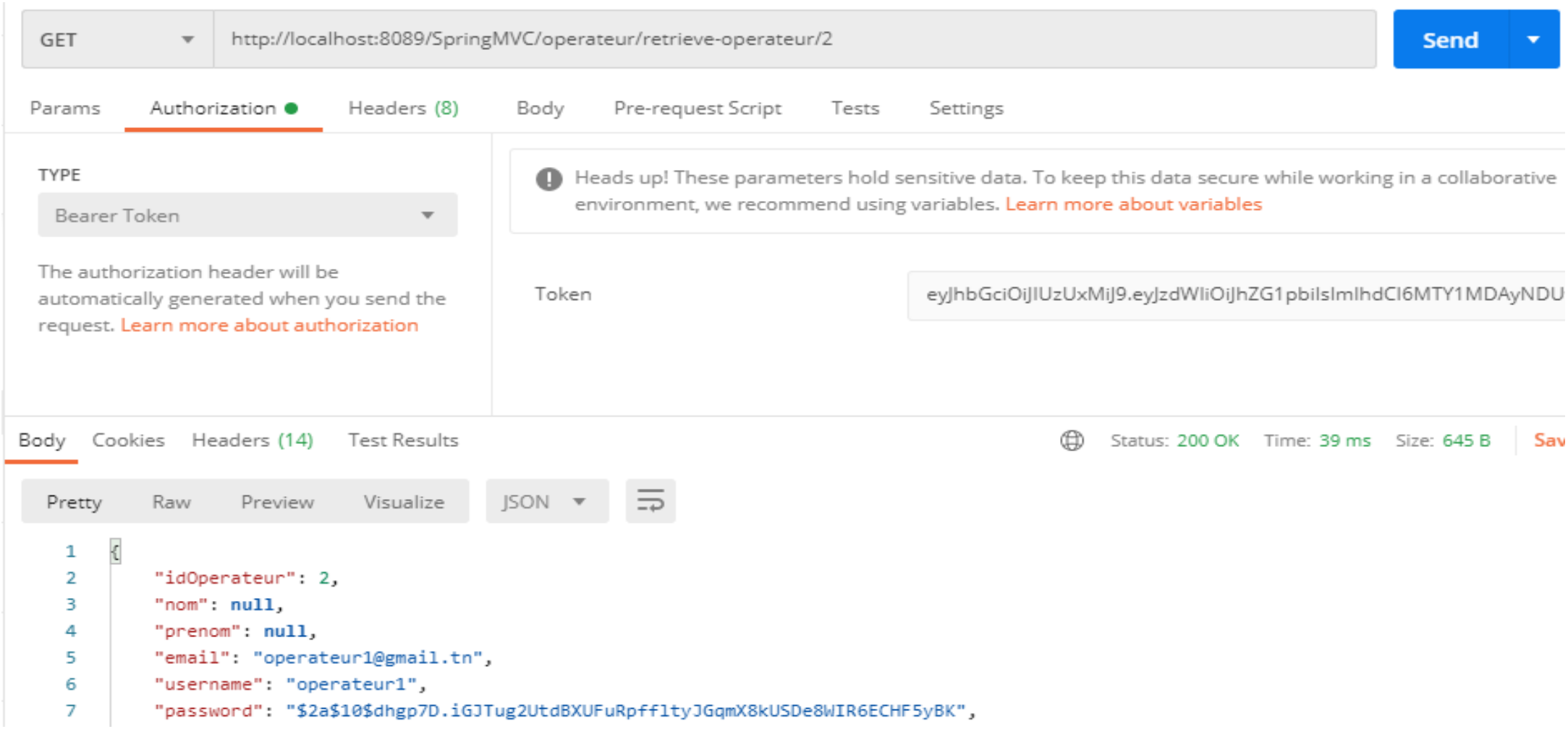
TP –Authentification

- Dans notre exemple, nous nous sommes basés sur la notion de JWT Token authentification. Le token récupéré nous permettra d'exécuter les services auxquels les acteurs auront droit



TP –Authentification

- Dans notre exemple, nous nous sommes basés sur la notion de JWT Token authentification. Le token récupéré nous permettra d'exécuter les services auquels les acteurs auront droit



Si vous avez des questions, n'hésitez pas à nous contacter :

Département Informatique
UP ASI
Bureau E204